Homework 11

Author: Manushri Pendekanti (mp79)

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
```

Question 1: Implementing K-means

Do **not** use a built-in K-means function for Q1–Q2. Generate 10 points from two independent standard normals; $K=2$; random nonempty initial assignment; report seed.

Question 1 — Part a

(Your written answer.)

In [2]:
```
# One iteration: update means, assign labels, plot

SEED = 42
print("Random seed:", SEED)

rng = np.random.default_rng(SEED)


X = rng.standard_normal((10, 2))
K = 2


initial_labels = np.concatenate((
    np.arange(K),
    rng.integers(0, K, size=len(X) - K)
))
rng.shuffle(initial_labels)


means = np.array([
    X[initial_labels == k].mean(axis=0)
    for k in range(K)
])


squared_distances = np.sum(
    (X[:, None, :] - means[None, :, :]) ** 2,
    axis=2
)
```

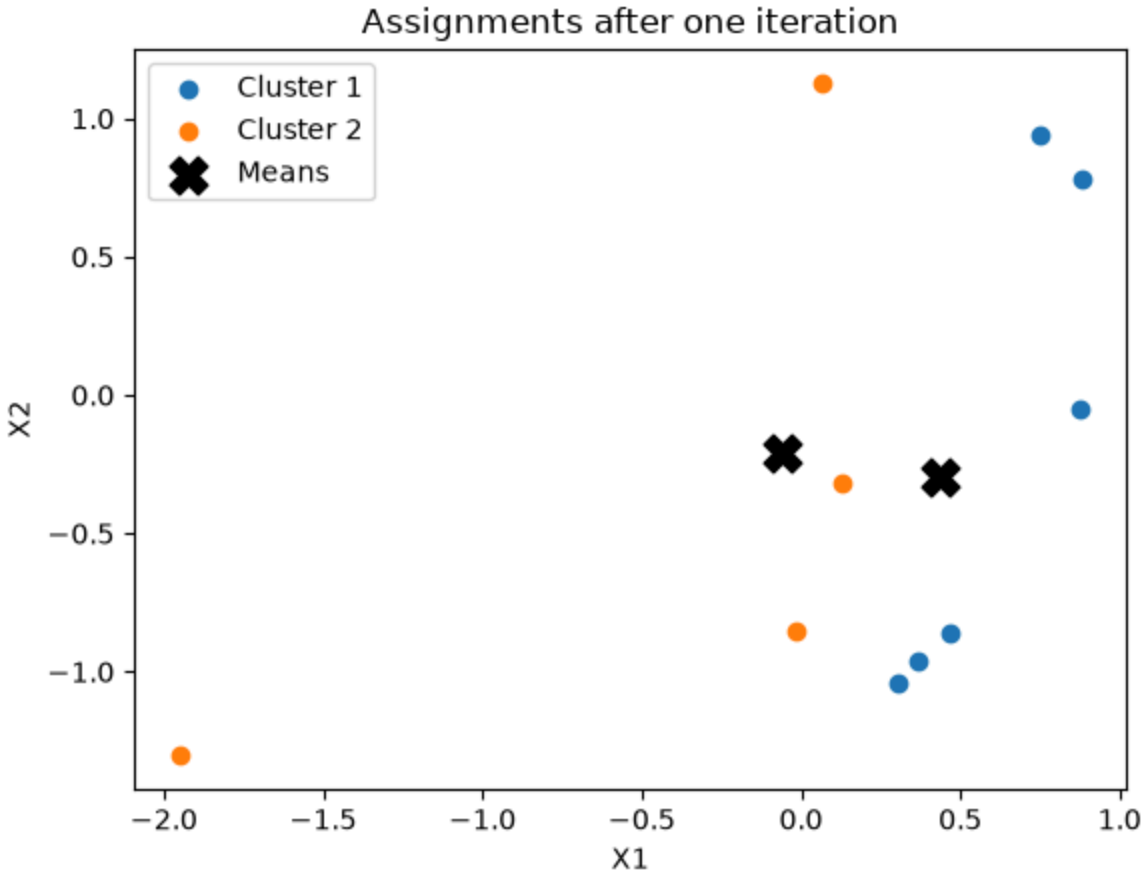
```python
updated_labels = np.argmin(squared_distances, axis=1)

print("Initial assignments:", initial_labels + 1)
print("Cluster means:\n", means)
print("Updated assignments:", updated_labels + 1)

for k in range(K):
    plt.scatter(
        X[updated_labels == k, 0],
        X[updated_labels == k, 1],
        label=f"Cluster {k + 1}"
    )

plt.scatter(
    means[:, 0], means[:, 1],
    marker="X", s=180, c="black", label="Means"
)
plt.xlabel("X1")
plt.ylabel("X2")
plt.title("Assignments after one iteration")
plt.legend()
plt.show()
```

```
Random seed: 42
Initial assignments: [1 2 2 1 1 1 2 2 2 1]
Cluster means:
 [[ 0.43472092 -0.29628092]
 [-0.05965863 -0.21050973]]
Updated assignments: [1 1 2 2 2 1 2 1 1 1]
```



Question 1 — Part b

(Your written answer.)

In [3]:

```python
# kmeans_custom(X, K, initial_labels, max_iter=...)

def kmeans(X, K, initial_labels, max_iter=100):
    X = np.asarray(X, dtype=float)
    initial_labels = np.asarray(initial_labels)

    if X.ndim != 2 or not np.all(np.isfinite(X)):
        raise ValueError("X must be a finite, two-dimensional data matrix.")

    n, p = X.shape

    if not isinstance(K, (int, np.integer)) or not 1 <= K <= n:
        raise ValueError("K must be an integer between 1 and n.")

    if not isinstance(max_iter, (int, np.integer)) or max_iter < 0:
        raise ValueError("max_iter must be a nonnegative integer.")

    if initial_labels.shape != (n,):
        raise ValueError("Provide one initial label per observation.")

    if not np.all(np.isin(initial_labels, np.arange(K))):
        raise ValueError("Labels must be integers from 0 to K - 1.")

    labels = initial_labels.astype(int).copy()

    if len(np.unique(labels)) != K:
        raise ValueError("Every cluster must initially be nonempty.")

    means = np.array([
        X[labels == k].mean(axis=0)
        for k in range(K)
    ])

    for iteration in range(max_iter):

        for k in range(K):
            if np.any(labels == k):
                means[k] = X[labels == k].mean(axis=0)

        # Assign observations to their closest means
        squared_distances = np.sum(
            (X[:, None, :] - means[None, :, :]) ** 2,
            axis=2
        )
        new_labels = np.argmin(squared_distances, axis=1)

        if np.array_equal(labels, new_labels):
            break

        labels = new_labels
```

```
    # Recalculate means using the final assignments
    for k in range(K):
        if np.any(labels == k):
            means[k] = X[labels == k].mean(axis=0)

    wcss = np.sum((X - means[labels]) ** 2)

    return labels, means, wcss
```

Question 1 — Part c

(Your written answer.)

In [4]:
```
# Two random starts; compare WCSS and groupings

def random_nonempty_labels(n, K, rng):
    labels = np.concatenate((
        np.arange(K),
        rng.integers(0, K, size=n - K)
    ))
    rng.shuffle(labels)
    return labels


rng1 = np.random.default_rng(10)
rng2 = np.random.default_rng(20)

initial1 = random_nonempty_labels(len(X), K, rng1)
initial2 = random_nonempty_labels(len(X), K, rng2)

while np.array_equal(
    initial1[:, None] == initial1[None, :],
    initial2[:, None] == initial2[None, :]
):
    initial2 = random_nonempty_labels(len(X), K, rng2)

labels1, means1, wcss1 = kmeans(X, K, initial1)
labels2, means2, wcss2 = kmeans(X, K, initial2)

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

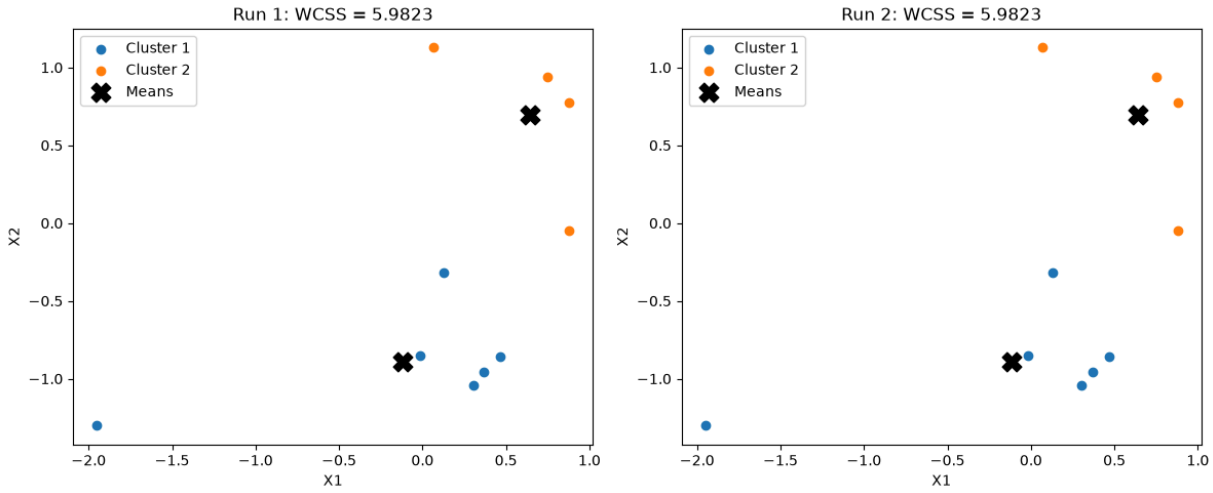
for ax, labels, means, wcss, run in [
    (axes[0], labels1, means1, wcss1, 1),
    (axes[1], labels2, means2, wcss2, 2)
]:
    for k in range(K):
        ax.scatter(
            X[labels == k, 0],
            X[labels == k, 1],
            label=f"Cluster {k + 1}"
        )

    ax.scatter(
        means[:, 0], means[:, 1],
```

```
        marker="X", s=180, c="black", label="Means"
    )
    ax.set_xlabel("X1")
    ax.set_ylabel("X2")
    ax.set_title(f"Run {run}: WCSS = {wcss:.4f}")
    ax.legend()

plt.tight_layout()
plt.show()

#both produced same grouping
#observations 1,3,4,5,8 and 9 were
#in cluster 1 and rest in cluster 2
#means were for 1 = -0.11 and 2 = 0.64
#they converged to same solution
```



Question 2: Image color compression

Use your K-means function on a color image (≤ 200×200 pixels, RGB).

Question 2 — Part a

(Your written answer.)

In [5]:
```
# Load image, plot, build n_pixels x 3 RGB matrix

from PIL import Image
from sklearn.datasets import load_sample_image

image = Image.fromarray(load_sample_image("flower.jpg"))
image.thumbnail((200, 200))

# Use RGB values on the 0–1 scale for every fit
image_array = np.asarray(image, dtype=float) / 255.0
height, width, channels = image_array.shape

plt.imshow(image_array)
plt.title("Original image")
```

```
plt.axis("off")
plt.show()

print("Image dimensions (height, width, channels):", image_array.shape)

pixels = image_array.reshape(-1, 3)

print("Pixel data matrix dimensions:", pixels.shape)
```




```
Image dimensions (height, width, channels): (133, 200, 3)
Pixel data matrix dimensions: (26600, 3)
```

Question 2 — Part b

(Your written answer.)

In [ ]:
```
# K = 2, 3, 5, 10: means, WCSS, reconstructed plots

K_values = [2, 3, 5, 10]
results = {}

fig, axes = plt.subplots(2, 2, figsize=(10, 8))

for ax, K in zip(axes.ravel(), K_values):
    rng = np.random.default_rng(42)


    initial_labels = np.concatenate((
        np.arange(K),
        rng.integers(0, K, size=len(pixels) - K)
    ))
    rng.shuffle(initial_labels)
```

```
    labels, means, wcss = kmeans(
        pixels, K, initial_labels, max_iter=100
    )

    compressed = means[labels].reshape(height, width, 3)

    results[K] = {
        "means": means,
        "wcss": wcss,
        "image": compressed
    }

    print(f"\nK = {K}")
    print("Cluster means (RGB scale: 0–1):")
    print(np.round(means, 4))
    print(f"Within-cluster sum of squares: {wcss:.4f}")

    ax.imshow(np.clip(compressed, 0, 1))
    ax.set_title(f"K = {K}, WCSS = {wcss:.2f}")
    ax.axis("off")

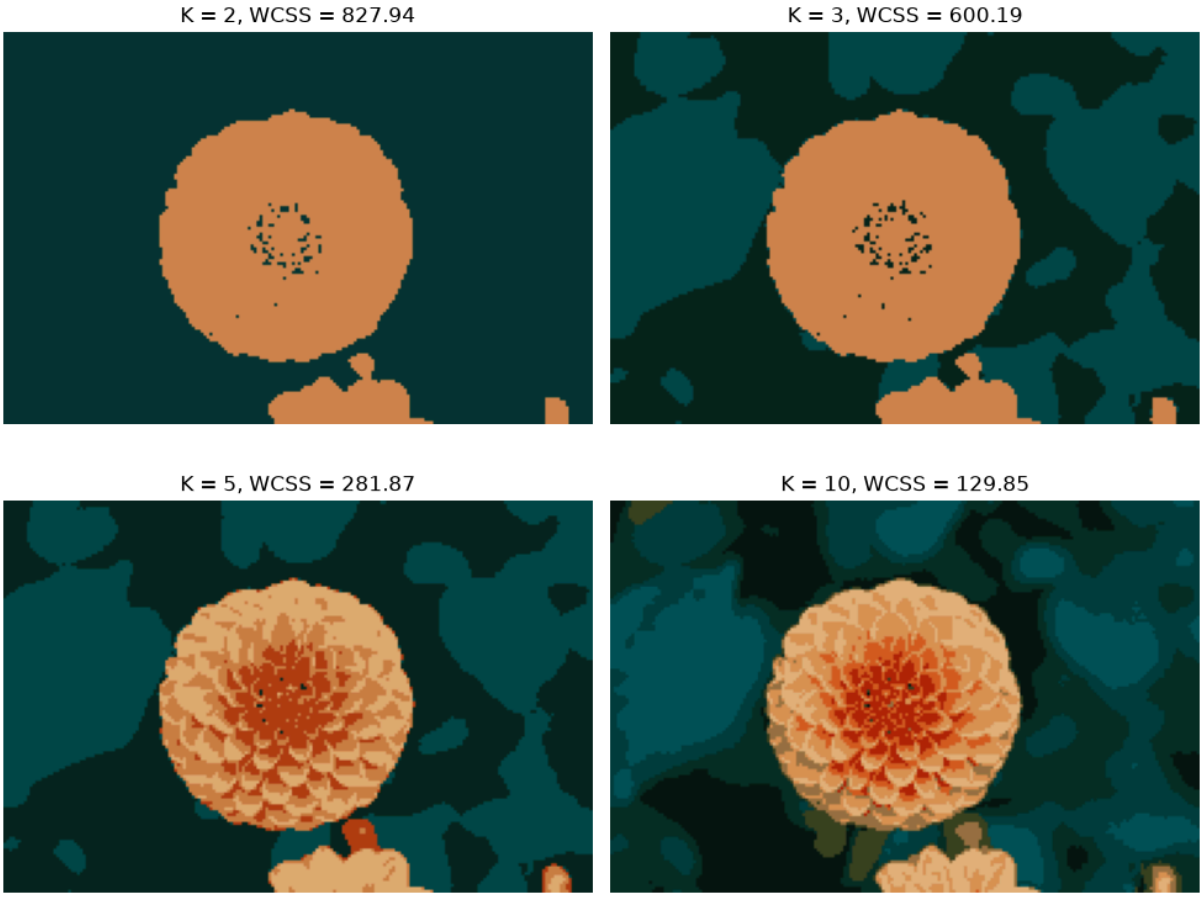
plt.tight_layout()
plt.show()
```

```
K = 2
Cluster means (RGB scale: 0–1):
[[0.0239 0.213  0.1964]
 [0.8191 0.5256 0.3088]]
Within-cluster sum of squares: 827.9393

K = 3
Cluster means (RGB scale: 0–1):
[[0.0375 0.1515 0.1158]
 [0.0111 0.2796 0.2838]
 [0.8204 0.5279 0.3101]]
Within-cluster sum of squares: 600.1929

K = 5
Cluster means (RGB scale: 0–1):
[[0.797  0.4998 0.2683]
 [0.8784 0.6686 0.4435]
 [0.0301 0.1522 0.1177]
 [0.6904 0.2427 0.0784]
 [0.0093 0.28   0.2851]]
Within-cluster sum of squares: 281.8683

K = 10
Cluster means (RGB scale: 0–1):
[[0.2318 0.2568 0.1314]
 [0.0026 0.32   0.3421]
 [0.689  0.144  0.036 ]
 [0.601  0.4507 0.2564]
 [0.8835 0.6973 0.4809]
 [0.0249 0.1823 0.1487]
 [0.0204 0.0939 0.066 ]
 [0.8501 0.5698 0.3296]
 [0.8254 0.3706 0.1356]
 [0.0072 0.2442 0.2379]]
Within-cluster sum of squares: 129.8455
```



Question 2 — Part c

I would choose k = 10 because it had lowest WCSS and it basically kept most of flower pic and then smaller K values. Also since smaller WCSS it closest to the true colors

Question 3: Hierarchical clustering of cars

Use `data/mtcars.csv`. Label with `car` only; cluster the 11 numeric variables.

Question 3 — Part a

(Your written answer.)

In [ ]:
```
# Standardize; single/complete/average linkage dendrograms

from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import linkage, dendrogram, cut_tree

cars = pd.read_csv("data/mtcars.csv")
X = cars.drop(columns="car")


scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
methods = ["single", "complete", "average"]
linkages = {}

fig, axes = plt.subplots(3, 1, figsize=(15, 18))

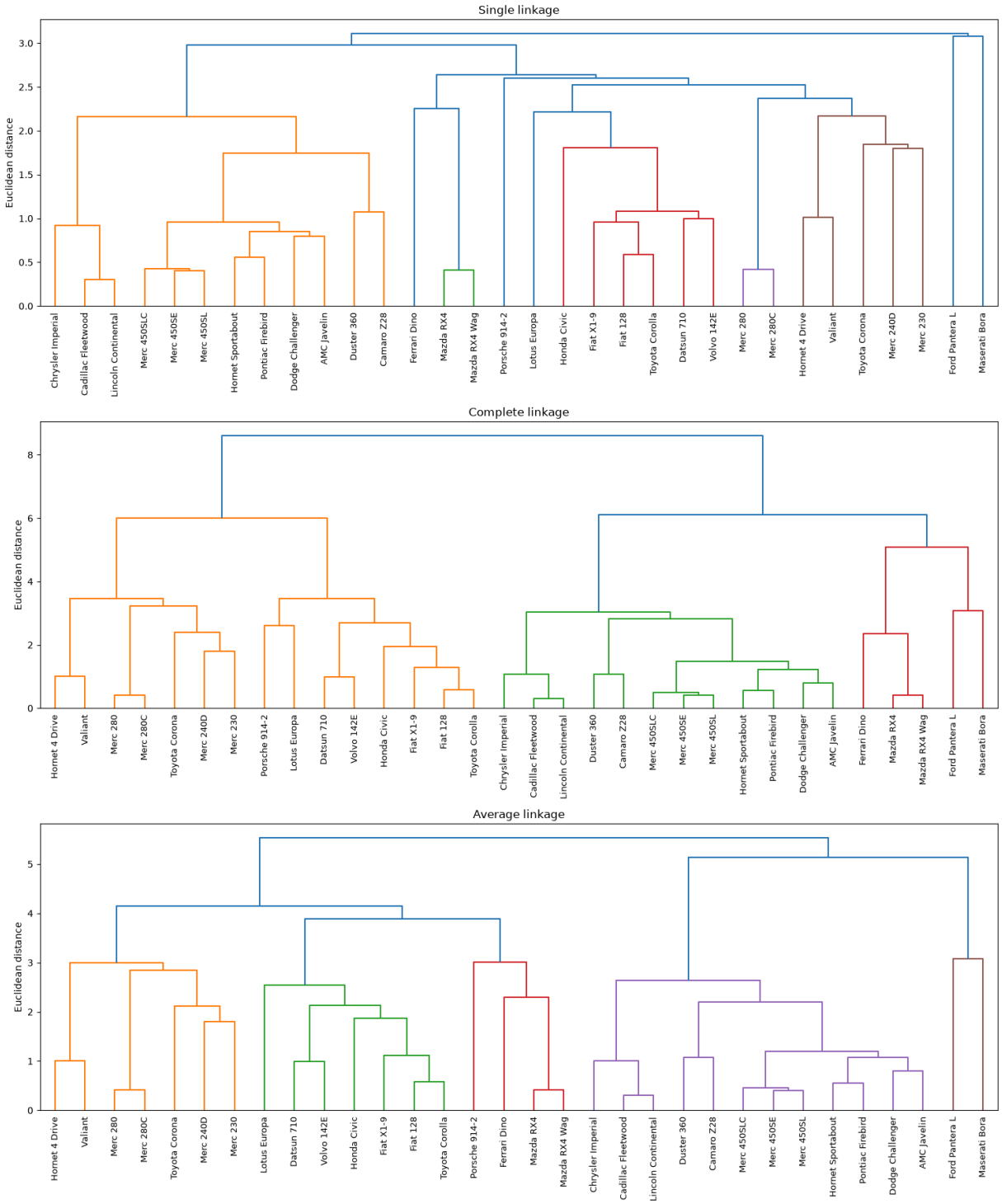
for ax, method in zip(axes, methods):
    Z = linkage(X_scaled, method=method, metric="euclidean")
    linkages[method] = Z

    dendrogram(
        Z,
        labels=cars["car"].to_numpy(),
        leaf_rotation=90,
        leaf_font_size=9,
        ax=ax
    )

    ax.set_title(f"{method.capitalize()} linkage")
    ax.set_ylabel("Euclidean distance")

plt.tight_layout()
plt.show()

#all of them identified similar cars
#single linakge showcased not a lot of difference among cars
#complete linkage there was more balanced groups
#average linakge it in the middle
```



Question 3 — Part b

(Your written answer.)

In [ ]:
```
# Chosen linkage + K; memberships; cluster means on raw scale

chosen_method = "complete"
K = 3

clusters = cut_tree(
    linkages[chosen_method],
```

```
    n_clusters=K
).ravel() + 1

cars_clustered = cars.copy()
cars_clustered["Cluster"] = clusters

# Report memberships
for k in range(1, K + 1):
    print(f"\nCluster {k}:")
    members = cars_clustered.loc[
        cars_clustered["Cluster"] == k, "car"
    ]
    print(members.to_string(index=False))


cluster_means = cars_clustered.groupby("Cluster")[X.columns].mean()
cluster_sizes = cars_clustered.groupby("Cluster").size()

summary = cluster_means.copy()
summary.insert(0, "Number of cars", cluster_sizes)

print("\nCluster means on the original scales:")
print(summary.round(2).to_string())

# Plot the chosen clustering
plt.figure(figsize=(15, 6))

dendrogram(
    linkages[chosen_method],
    labels=cars["car"].to_numpy(),
    leaf_rotation=90,
    leaf_font_size=9
)

Z = linkages[chosen_method]
cut_height = (Z[-K, 2] + Z[-K + 1, 2]) / 2

plt.axhline(
    cut_height,
    color="red",
    linestyle="--",
    label=f"Cut for {K} clusters"
)

plt.title("Complete linkage with three clusters")
plt.ylabel("Euclidean distance")
plt.legend()
plt.tight_layout()
plt.show()

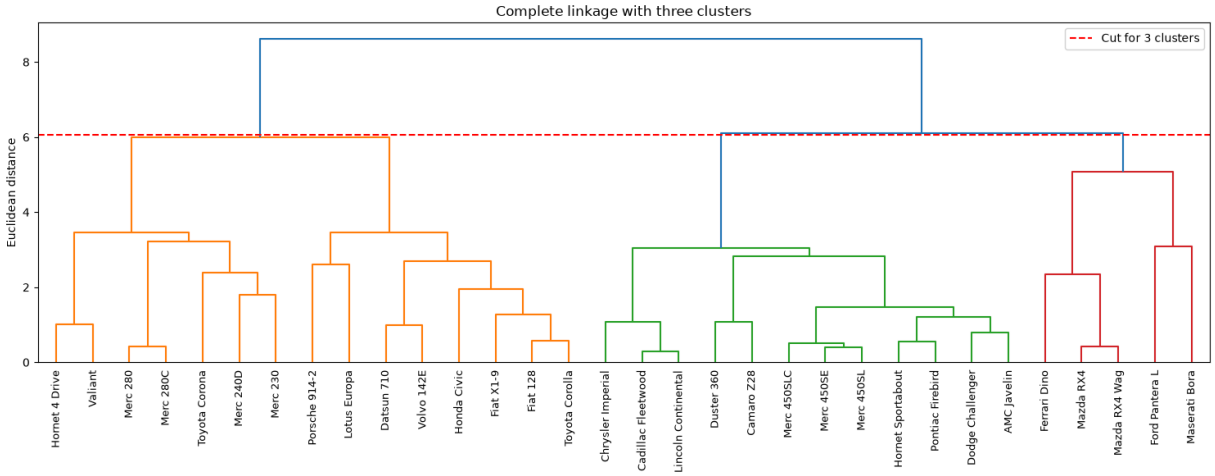
#I would choose complete linkage
#cluster 1 focuses on csrs best performance
#cluster 2 more fuel efficnet cars
#cluster 3 automatic transmissions cars
#standardizing means any vars that have large scales prevent
#dominating or taking over the distances
```

```
Cluster 1:
     Mazda RX4
 Mazda RX4 Wag
Ford Pantera L
  Ferrari Dino
 Maserati Bora

Cluster 2:
    Datsun 710
Hornet 4 Drive
       Valiant
     Merc 240D
      Merc 230
      Merc 280
     Merc 280C
      Fiat 128
   Honda Civic
Toyota Corolla
 Toyota Corona
     Fiat X1-9
 Porsche 914-2
  Lotus Europa
    Volvo 142E

Cluster 3:
  Hornet Sportabout
         Duster 360
         Merc 450SE
         Merc 450SL
        Merc 450SLC
 Cadillac Fleetwood
Lincoln Continental
  Chrysler Imperial
   Dodge Challenger
        AMC Javelin
         Camaro Z28
   Pontiac Firebird

Cluster means on the original scales:
         Number of cars    mpg   cyl    disp      hp  drat    wt   qsec    v
s    am  gear  carb
Cluster
1                     5  18.50  6.80  223.40  198.80  3.84  3.00  15.62  0.0
0  1.00  4.60  5.20
2                    15  24.65  4.53  131.65   91.33  3.90  2.58  19.16  0.9
3  0.53  3.93  1.80
3                    12  15.05  8.00  357.62  194.17  3.12  4.10  17.14  0.0
0  0.00  3.00  3.08
```


Complete linkage with three clusters
Cut for 3 clusters
Euclidean distance
0
2
4
6
8
Hornet 4 Drive
Valiant
Merc 280
Merc 280C
Toyota Corona
Merc 240D
Merc 230
Porsche 914-2
Lotus Europa
Datsun 710
Volvo 142E
Honda Civic
Fiat X1-9
Fiat 128
Toyota Corolla
Chrysler Imperial
Cadillac Fleetwood
Lincoln Continental
Duster 360
Camaro Z28
Merc 450SLC
Merc 450SE
Merc 450SL
Hornet Sportabout
Pontiac Firebird
Dodge Challenger
AMC Javelin
Ferrari Dino
Mazda RX4
Mazda RX4 Wag
Ford Pantera L
Maserati Bora